

UCS503 – Software Engineering

BloodLine

Predictive Blood Shortage and Smart Donor Matching Platform

Submitted by:

1024030287 Avraj Singh Sekhon

1024030288 Manya Sabherwal

1024030296 Krish Kumar

1024030303 Kyna Sood

Group: 3C22 Third Year, CSE

Submitted to:

Dr. Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Context	1
1.1.1	Problem Statement	1
1.2	Project Objectives	1
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	10
2.2	Implementation Details	11
2.2.1	User Roles	11
2.2.2	Database Design	11
2.2.3	Module 1: Core Functionality	13
2.2.4	Module 2: Secondary Features	13
2.2.5	Dashboard	14
3	Results and Evaluation	15
3.1	Testing Methodology	15
3.2	Performance Metrics	15
3.3	Analysis	15
4	Conclusion	17
4.1	Summary of Work	17
4.2	Key Findings	17
4.3	Future Work and Recommendations	17
4.4	Final Remarks	18
	Bibliography	19

Chapter 1

Introduction

1.1 Background and Context

Blood banks and hospitals routinely face uncertainty in maintaining adequate blood availability. Current stock levels can be observed after the fact, but the short-term risk of an approaching shortage is rarely estimated in advance, and inventory records, hospital request data, and potential donor pools tend to exist in isolation from one another. This slows down coordination exactly when speed matters most. BloodLine is a Software Engineering project undertaken to close this gap by combining inventory tracking, near-term demand forecasting, shortage-risk detection, and donor matching into a single role-based platform.

1.1.1 Problem Statement

The specific pain points motivating this project are:

- **Reactive shortage management:** current stock can be monitored, but there is no systematic early warning for an approaching shortage.
- **Fragmented coordination:** inventory, hospital blood requests, and donor outreach are not unified into a single workflow.
- **Inefficient donor outreach:** identifying suitable donors by blood-group compatibility, availability, donation cooldown, and consent is otherwise manual and time-consuming.

1.2 Project Objectives

1. Provide role-based dashboards for donors, hospital coordinators, blood bank managers, inventory staff, medical staff, and system administrators.
2. Track blood inventory by blood group and component with a full transaction history.
3. Generate a short-term demand estimate from historical (currently synthetic) demand data.
4. Detect and classify shortage risk (LOW / MODERATE / HIGH) using a transparent, explainable rule.
5. Identify potential donor matches using compatibility, availability, consent, and cooldown rules.

6. Support an end-to-end hospital blood-request workflow, from submission through manager review to inventory issue.

This report documents the BloodLine implementation as it currently stands. It does not claim features, models, or metrics that are not present in the submitted source code; where a diagram or an earlier proposal describes something ahead of what is currently implemented, this is flagged explicitly rather than left unstated.

Methodology and Design

2.1 System Architecture

BloodLine is designed as a role-based, layered web application that integrates blood inventory management, hospital blood requests, donor management, demand forecasting, shortage-risk detection, medical screening, and reporting within a single system. The architecture separates the user-facing frontend, backend application services, and persistent database layer, allowing each component to perform a distinct responsibility while communicating through defined interfaces. The system supports six user roles – Donor, Hospital Coordinator, Blood Bank Manager, Inventory Staff, Medical Staff, and System Administrator – with access controlled according to their respective responsibilities. The following sections describe the overall architecture and its detailed data flows, interactions, and system behaviour.

2.1.1 High-Level Design

BloodLine follows a simple layered architecture: six role-based user groups access a single-page web frontend over HTTP, which communicates with a FastAPI backend over a JSON REST API, which in turn reads and writes a SQLite database through a set of application-service modules (Figure 2.1).

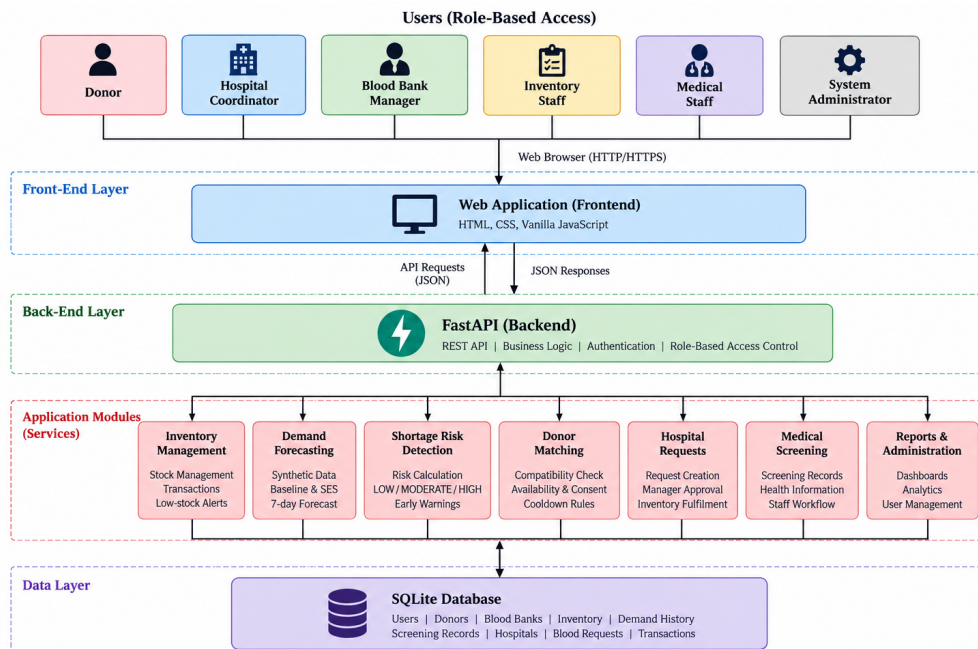


Figure 2.1: High-Level Architecture of the BloodLine System

The Level 0 data flow diagram treats BloodLine as a single process exchanging data with the six external actors described above (Figure 2.2).

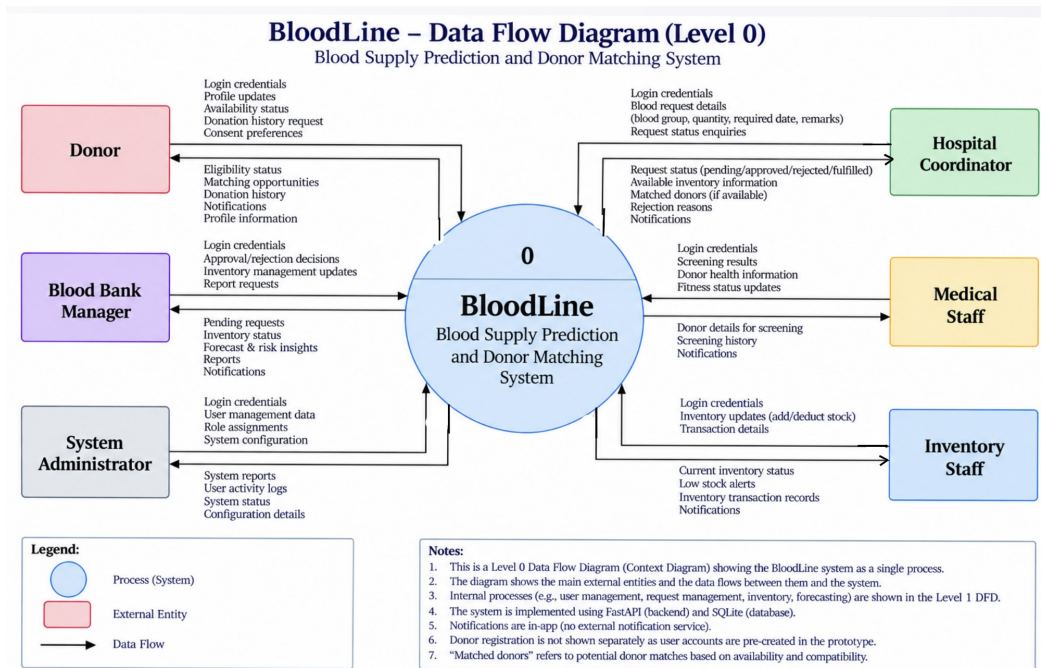


Figure 2.1: Level 0 Data Flow Diagram (Context Diagram) for the BloodLine System

Figure 2.2: Level 0 Data Flow Diagram (Context Diagram)

The Level 1 data flow diagram decomposes the system into six internal processes – User Management, Blood Request Management, Inventory Management, Donor & Screening Management, Demand Forecasting & Risk Analysis, and Reporting & Dashboards – each reading from and writing to a grouping of the underlying database tables (Figure 2.3).

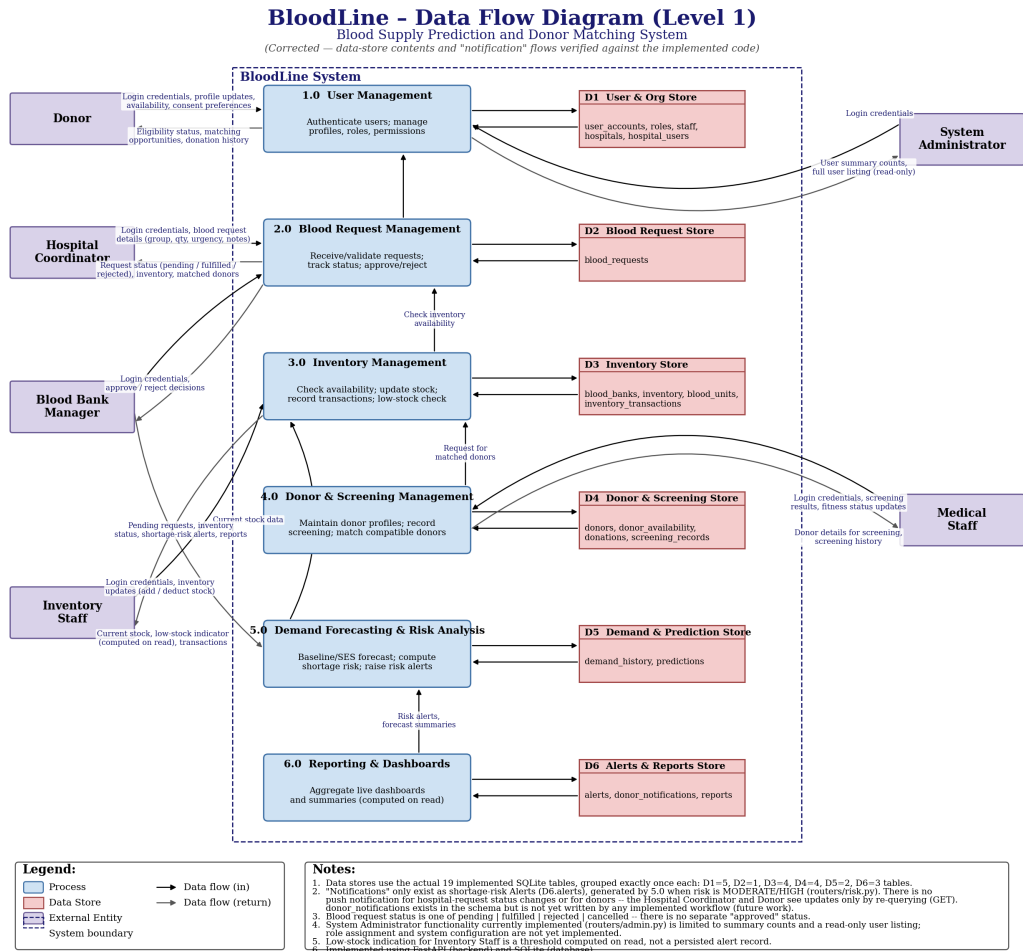


Figure 2.3: Level 1 Data Flow Diagram

The use-case diagram groups functionality by the six implemented roles. Every role-specific use case requires authentication, and *View Dashboard* and *Update Profile* are common to all authenticated users (Figure 2.4).

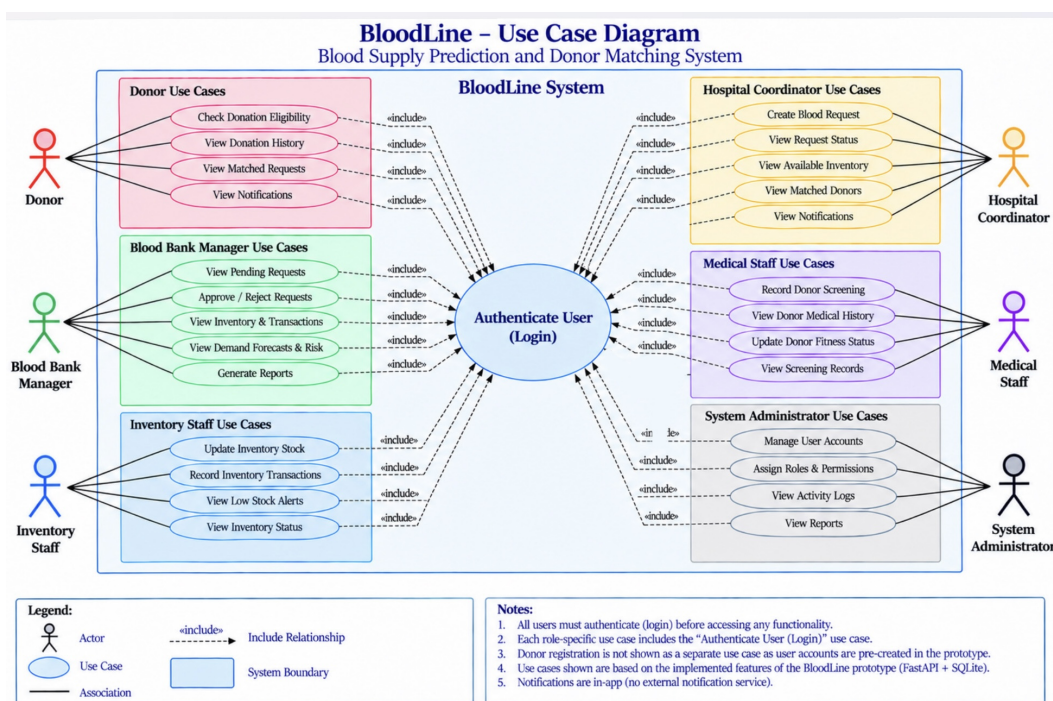


Figure 2.2: Use Case Diagram for the Bloodline System

Figure 2.4: Use Case Diagram

The UML class diagram below is drawn directly from the actual implemented SQLAlchemy schema and supersedes all earlier versions of this diagram. It reflects the 19 tables described in Section 2.2.2 and only the foreign-key relationships that are genuinely present in the models (Figure 2.5).

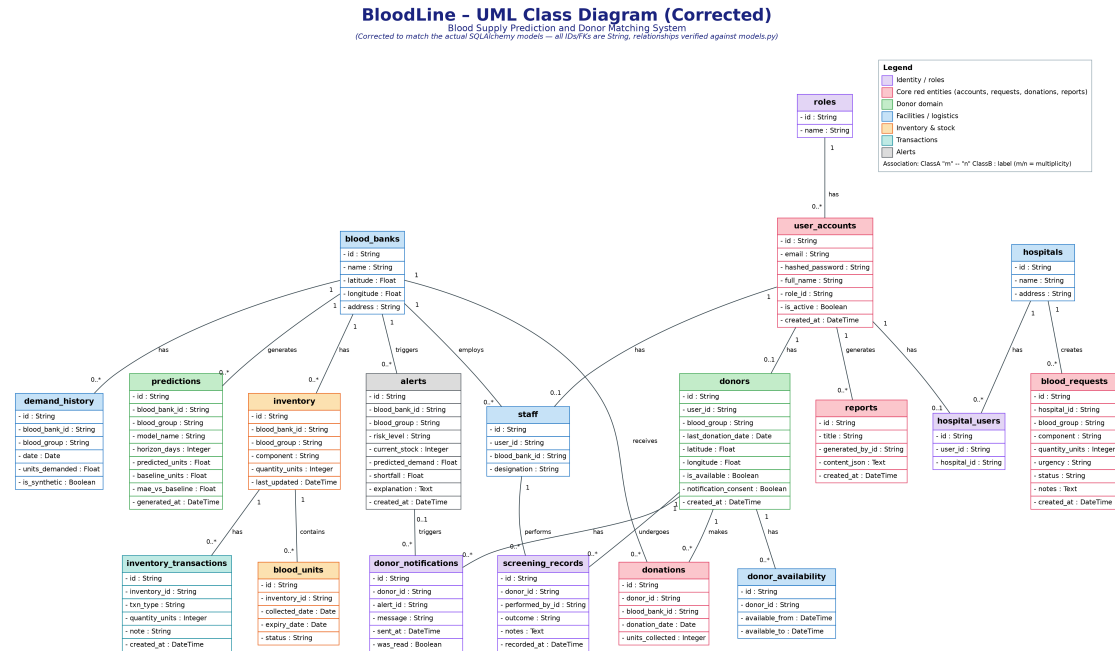


Figure 2.5: UML Class Diagram (Corrected, Based on Actual Database Schema)

The activity diagram traces the hospital blood-request workflow – the main integrated flow spanning the Hospital Coordinator, the BloodLine system, the Blood Bank Manager, and the inventory subsystem – exactly as implemented. There is no separate persisted “Approved” status, and no notification is generated for this workflow (Figure 2.6).

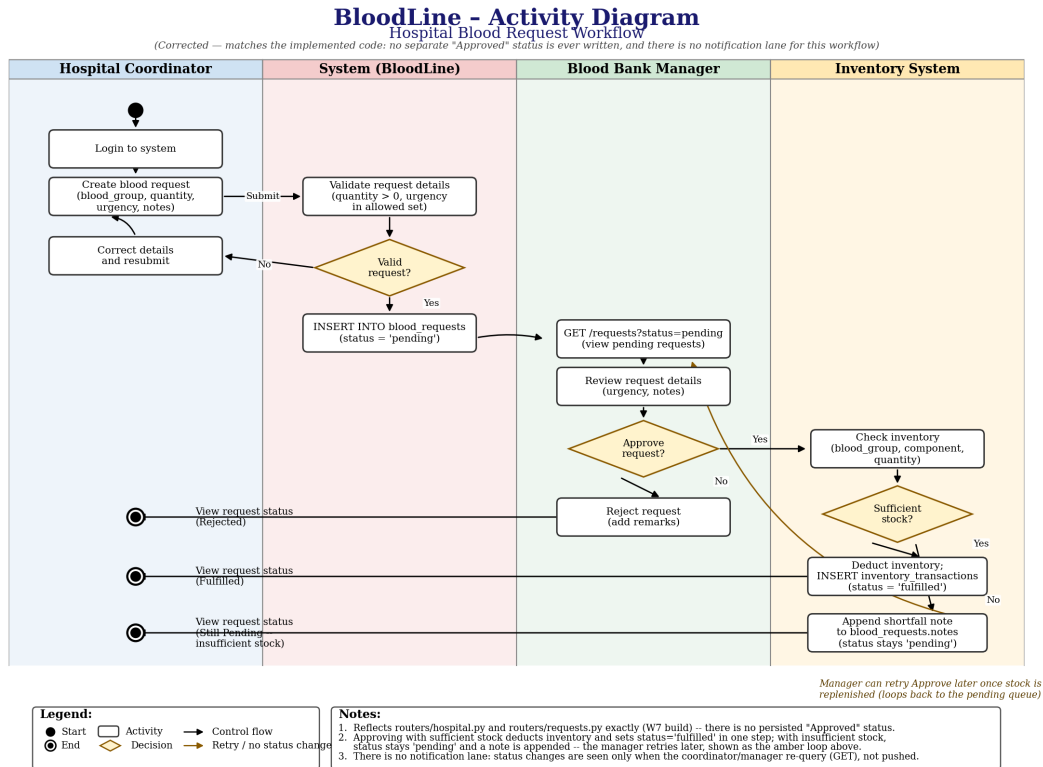


Figure 2.6: Activity Diagram – Hospital Blood Request Workflow

The state diagram represents the blood-request lifecycle exactly as implemented: the stored `status` column only ever takes the value `pending`, `fulfilled`, `rejected`, or `cancelled`. An approval attempt made against insufficient stock is drawn as a self-loop on `Pending`, since a note is appended but the status does not change (Figure 2.7).

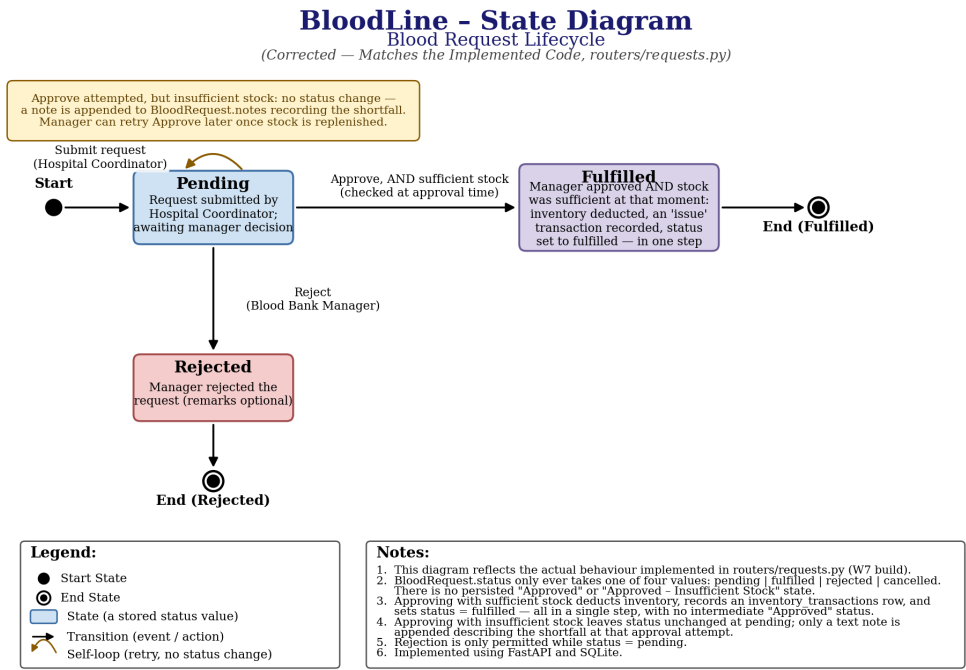


Figure 2.7: State Diagram – Blood Request Lifecycle

The sequence diagram covers the hospital request → approval → inventory-issue flow, including the rejected, sufficient-stock, and insufficient-stock alternative paths, using the actual endpoint and table names from the implemented code. There is no separate notification module in this workflow; the manager and the hospital coordinator each learn of a status change only by re-querying the relevant endpoint (Figure 2.8).

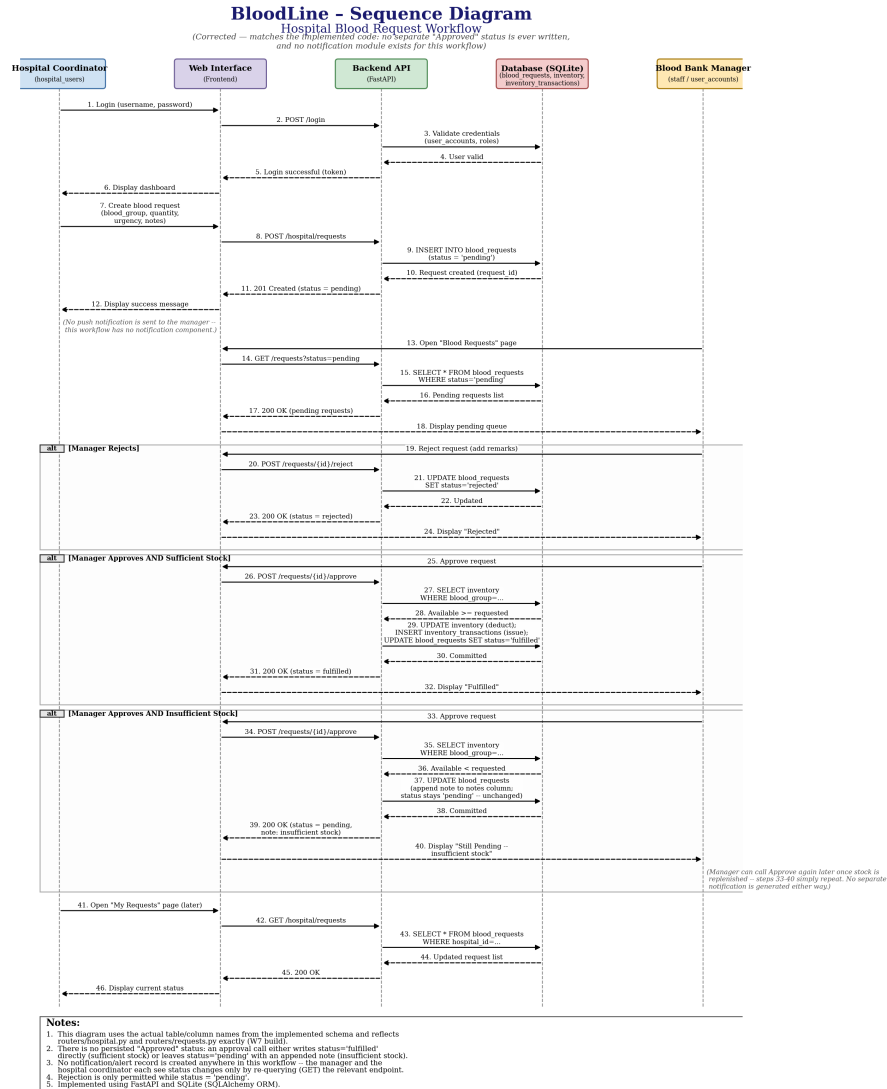


Figure 2.8: Sequence Diagram – Hospital Blood Request Workflow

2.1.2 Technical Stack

The implemented stack, verified directly against the submitted source code, is summarised in Table 2.1.

The original project proposal envisaged a React with Tailwind CSS frontend and a PostgreSQL database. The current implementation instead uses a single vanilla HTML/CSS/JavaScript page and SQLite, so that the running prototype has no build toolchain and no external database service to configure during development and demonstration. Because the back-end exposes a REST API, both substitutions can be made later without changing the API contract.

Category	Technology / Tool	Purpose
Frontend	HTML, CSS, Vanilla JavaScript	Static web interface and client-side interaction
Backend	Python, FastAPI	REST API and server-side application logic
Database	SQLite	Persistent local data storage
ORM	SQLAlchemy	Database access and object-relational mapping
API	REST / JSON	Frontend-backend communication
Authentication	Role-Based Access Control	Role-specific authorization
Development	VS Code / Terminal	Development and local execution
API Testing	FastAPI Swagger / OpenAPI	API testing and endpoint verification

Table 2.1: Implemented technical stack

2.2 Implementation Details

2.2.1 User Roles

Six roles are implemented with distinct permissions, enforced through a shared role-based access dependency on every protected API route: Donor, Hospital Coordinator, Blood Bank Manager, Inventory Staff, Medical Staff, and System Administrator.

2.2.2 Database Design

The implemented schema contains exactly 19 tables:

1. alerts
2. blood_banks
3. blood_requests
4. blood_units
5. demand_history
6. donations
7. donor_availability
8. donor_notifications
9. donors
10. hospital_users
11. hospitals
12. inventory
13. inventory_transactions
14. predictions

15. reports
16. roles
17. screening_records
18. staff
19. user_accounts

The supported foreign-key relationships, verified against the SQLAlchemy models, are:

- user_accounts.role_id → roles.id
- donors.user_id → user_accounts.id
- donor_availability.donor_id → donors.id
- donations.donor_id → donors.id
- donations.blood_bank_id → blood_banks.id
- hospital_users.user_id → user_accounts.id
- hospital_users.hospital_id → hospitals.id
- blood_requests.hospital_id → hospitals.id
- blood_units.inventory_id → inventory.id
- inventory.blood_bank_id → blood_banks.id
- inventory_transactions.inventory_id → inventory.id
- demand_history.blood_bank_id → blood_banks.id
- predictions.blood_bank_id → blood_banks.id
- screening_records.donor_id → donors.id
- screening_records.performed_by_id → staff.id
- staff.user_id → user_accounts.id
- staff.blood_bank_id → blood_banks.id
- donor_notifications.donor_id → donors.id
- donor_notifications.alert_id → alerts.id
- alerts.blood_bank_id → blood_banks.id
- reports.generated_by_id → user_accounts.id

Not every table is actively read from or written to by the current feature set: **donor_availability**, **donor_notifications**, and **reports** exist in the schema so that it will not need to change shape when the corresponding later-phase features (fine-grained availability windows, targeted push notifications, saved/exported reports) are built, but they are not yet populated by any implemented workflow.

2.2.3 Module 1: Core Functionality

Role-based authentication. Users authenticate against `user_accounts`; every subsequent request is authorized against the caller’s role.

Inventory management. Current stock is tracked per blood bank, blood group, and component, with recorded transactions of type receipt, issue, expiry, and discard. Validation rejects zero, negative, or over-issue transaction quantities.

Synthetic demand generation. Because granular real-world blood-bank consumption data is not available for this prototype, demand is generated synthetically by combining a blood-group baseline, a weekly pattern, a seasonal effect, a holiday/event bump, emergency spikes, and random noise. The generator produces 180 days of history across 8 blood groups, giving 1,440 synthetic demand records per seeded blood bank. Every such record is tagged as synthetic, and this is surfaced wherever the data is shown.

Demand forecasting. The implemented forecasting approach does not use Prophet or SARIMA. Two candidate methods are computed: a 14-day naive baseline, and Simple Exponential Smoothing (SES) with $\alpha = 0.3$. A rolling-origin backtest over a recent holdout window computes the Mean Absolute Error (MAE) of both candidates, and the lower-MAE model is selected and reported alongside both MAE values.

Shortage-risk detection. A transparent rule layer, not a black-box classifier, computes:

$$\text{shortfall} = \text{predicted demand} - \text{current stock}$$

- LOW if $\text{shortfall} \leq 0$
- MODERATE if shortfall is positive and $\leq 25\%$ of current stock
- HIGH if shortfall is greater than 25% of current stock, or if current stock is zero while predicted demand is positive

2.2.4 Module 2: Secondary Features

Donor matching. Candidates are filtered by blood-group compatibility, availability, notification consent, and a 90-day donation cooldown, then ranked by a simple additive score. The module only surfaces potential matches: it does not provide medical clearance, and the final medical decision remains with medical staff.

Hospital blood request workflow. A Hospital Coordinator submits a request (blood group, component, quantity, urgency, notes), which enters the pending state. A Blood Bank Manager then approves or rejects it. On approval with sufficient stock, inventory is deducted, an issue transaction is recorded, and the request becomes fulfilled. On approval with insufficient stock, the request remains pending and an insufficient-stock note is appended to the request; there is no separate persisted status such as “Approved – Insufficient Stock”. On rejection, the request becomes rejected.

Medical screening. Medical staff can view a worklist of eligible donors, record a screening outcome, and view a donor’s screening history. Every screening record is tied to a specific staff member.

Reporting and dashboards. Reports aggregate inventory transactions and rerun the forecast/risk functions to produce live summaries; these are computed on read rather than persisted.

In-app notifications. Notifications are surfaced within the application UI only; no external push-notification service is integrated.

2.2.5 Dashboard

Dashboard. The Blood Bank Manager dashboard provides a consolidated view of the current operational state of the system. It presents inventory information by blood group, shortage-risk indicators, demand and forecast information, pending hospital requests, and relevant operational summaries. The dashboard obtains these summaries from the implemented inventory, forecasting, risk-analysis, and request-management functionality, allowing the manager to review the current situation from a single interface. Reports and dashboard summaries are computed from current system data rather than relying on separately persisted report results.

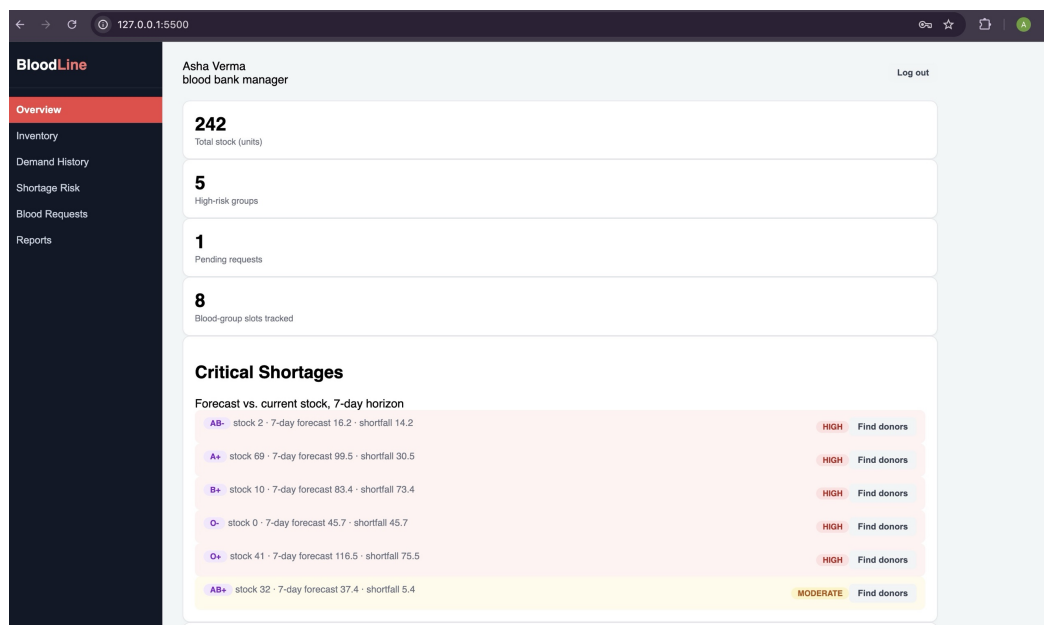


Figure 2.9: Blood Bank Manager Dashboard

Results and Evaluation

3.1 Testing Methodology

At this stage, testing is functional rather than performance-based:

- **Manual functional testing:** every role was logged into and every implemented page/action was exercised against seeded demo data.
- **Boundary/validation testing:** inventory transactions were tested with zero, negative, and over-issue quantities to confirm they are rejected.
- **Workflow testing:** the full hospital-request path (submit → approve/reject → inventory issue) was exercised for both the sufficient-stock and insufficient-stock cases.

No dedicated automated integration test suite or load/performance testing has been carried out at this stage; this is noted honestly rather than represented as complete.

3.2 Performance Metrics

No accuracy, precision/recall, response-time, throughput, or uptime figures are reported here, as no such measurements currently exist for this prototype. Table 3.1 instead reports functional verification against the implemented behaviour, which is the meaningful measure of progress at this stage.

3.3 Analysis

The six objectives stated in Section 1.2 each have at least a working implementation: role-based dashboards, inventory tracking, demand estimation, shortage-risk classification, donor matching, and the end-to-end hospital request workflow are all functional against seeded demo data. The main gap between the original proposal and the current build is the fidelity of individual components rather than missing functionality – the forecasting model, frontend framework, and database engine are all simpler than originally scoped, by deliberate engineering choice (Section 4.3). A notable implementation detail was keeping the manager approval path correct under partial stock: this is handled by leaving the request in a clearly-flagged pending state with a note, rather than either silently failing or incorrectly marking it fulfilled.

Verified Behaviour	Status
Login succeeds for all six roles with correct permission scoping	Verified
Inventory receipt/issue/expiry/discard transactions update stock correctly	Verified
Invalid (zero, negative, over-issue) transaction quantities are rejected	Verified
180-day synthetic demand series generated per blood group, labelled synthetic	Verified
Forecast generated with baseline and SES candidates, both MAE values reported	Verified
Shortage risk classified LOW/MODERATE/HIGH with plain-language explanation	Verified
Donor matching returns ranked, compatible, cooldown-cleared candidates	Verified
Hospital request submission appears in manager's pending queue	Verified
Approval with sufficient stock deducts inventory and marks request fulfilled	Verified
Approval with insufficient stock keeps request pending with a note	Verified
Screening outcome persists and reflects immediately in worklist/history	Verified
Reports display live-computed inventory, risk, and request summaries	Verified

Table 3.1: Functional verification summary against the implemented prototype

Conclusion

4.1 Summary of Work

BloodLine, as currently implemented, covers all six user roles, inventory management with transaction validation, synthetic demand generation, a transparent forecasting and shortage-risk pipeline, rule-based donor matching, and an end-to-end hospital blood-request workflow tying inventory, approvals, and screening together.

4.2 Key Findings

- A simple, auditable forecasting approach (naive baseline versus SES, chosen by backtested MAE) is sufficient to drive a working shortage-risk pipeline without requiring Prophet or SARIMA at this stage.
- Keeping the frontend as a single vanilla-JavaScript page removes an entire class of build/deployment risk, at no cost to the API design.
- Modelling the full 19-table schema up front meant that later features (hospital requests, screening, administration) were additive, requiring no schema migrations or breaking changes.

4.3 Future Work and Recommendations

- Evaluate Prophet or SARIMA (or another stronger model) against the current naive/SES baseline.
- Implement targeted donor push notifications (the `donor_notifications` table exists but is not yet wired into any workflow).
- Add a regional, multi-bank surplus and redistribution view using the existing donor/blood-bank location fields.
- Support finer-grained donor availability windows in place of the current on/off toggle.
- Migrate from SQLite to a managed PostgreSQL database for production use.
- Harden the deployment pipeline with CI/CD.
- Add saved/exportable PDF reports (the `reports` table exists but reports are currently computed live rather than persisted).

- Pursue authorized integration with real-world blood-demand data to replace the synthetic generator.
- Consider real-world deployment requirements, including data protection and access-control auditing appropriate to healthcare data.

4.4 Final Remarks

The implementation intentionally keeps its current scope honest against the original proposal: synthetic data is clearly labelled, an interpretable forecasting/risk approach is used in place of an unimplemented model, and deployment, notification, and regional-redistribution features are deferred to future iterations. This gives a working, demonstrable system to build on for the remainder of the course, with a clear and documented path to the originally proposed feature set.

Bibliography

- [Pres14] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.
- [Somm16] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [FastAPI] FastAPI Documentation. <https://fastapi.tiangolo.com/>.
- [SQLAlch] SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>.
- [SQLite] SQLite Documentation. <https://www.sqlite.org/docs.html>.
- [Hyndman21] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.
- [Gardner85] E. S. Gardner Jr., “Exponential Smoothing: The State of the Art,” *Journal of Forecasting*, vol. 4, no. 1, pp. 1–28, 1985.
- [WHO17] World Health Organization, *Blood Donor Selection: Guidelines on Assessing Donor Suitability for Blood Donation*, WHO Press, 2012.